

Week 1 Progress Summary: Problem Solving & Algorithms

Overview

This week focused on foundational problem-solving patterns, including string manipulation, two-pointer sliding windows, greedy approximations, and Kadane's algorithm. A total of 10 problems were successfully implemented in C++.

Problem Summaries

1. Beautiful Year

- Core Concept: Brute Force / Digit Extraction
- Approach: The program increments the starting year n by 1 and checks subsequent years sequentially. For each year, it extracts all four individual digits and uses a conditional statement to verify if all digits are mutually distinct. It prints the first year that satisfies this condition and terminates.

2. Books

- Core Concept: Two-Pointer Approach / Sliding Window
- Approach: Designed to find the maximum number of books that can be read within a time limit t . It maintains a variable window defined by left and right pointers. The right pointer expands the window by adding books to the sum. If sum exceeds t , the left pointer contracts the window from the left until the condition is re-satisfied. The maximum window size is tracked throughout.

3. Flipping Game

- Core Concept: Kadane's Algorithm / Dynamic Programming
- Approach: To maximize the number of 1s after exactly one subarray flip, the problem is transformed into finding the maximum subarray sum. Elements are mapped such that 0 becomes +1 (potential gain) and 1 becomes -1 (potential loss). Kadane's algorithm is then executed on this modified array to find the optimal segment to flip.

4. George and Accommodation

- Core Concept: Basic Conditional Filtering
- Approach: Iterates through n rooms, reading the current number of occupants a and the total capacity b . It increments a counter whenever a room has at least 2 free spaces remaining ($b - a > 1$) to accommodate George and his friend.

5. Lucky Sum of Digits

- Core Concept: Greedy Search / Digit Construction
- Approach: Finds the lexicographically smallest number consisting only of digits 4 and 7 that sums exactly to n . The loop dynamically prioritizes subtracting 7 when divisible, then 4 when divisible, and falls back to a combined step if needed. It then prints the constructed sequence of 4s followed by 7s.

6. Petya and Strings

- Core Concept: String Comparison & Normalization
- Approach: Normalizes two strings by converting all uppercase characters to lowercase via ASCII shifts (+32). After checking that the lengths match, it performs a lexicographical comparison using standard operators to output 0 if equal, 1 if the first is greater, and -1 if the second is greater.

7. Queue at the School

- Core Concept: Simulation
- Approach: Simulates a queue shuffle over t time steps. In each step, it passes through the string and swaps adjacent characters whenever a boy (B) is directly in front of a girl (G). The loop index is incremented an extra time upon a swap to prevent a single student from moving forward multiple positions in a single time step.

8. Regular Bracket Sequence

- Core Concept: Combinatorics / Validation logic
- Approach: Evaluates whether counts of specific bracket pairs ($((,)(, (),))$) can form a valid bracket sequence. It verifies that the count of open-pair blocks matches the close-pair blocks, and handles the edge case where independent middle pairs $()()$ exist without any outer enclosing brackets.

9. Stones on the Table

- Core Concept: String Traversal / Neighbor Checking
- Approach: Solves the problem of finding the minimum number of stones to remove so that no two adjacent stones have the same color. It checks the string linearly from index 0 to $n-2$ and increments a removal counter whenever $s[i] == s[i+1]$.

10. Translation

- Core Concept: Two-Pointer String Reversal
- Approach: Validates if string t is the reverse of string s . It manually reverses string s in-place by swapping symmetric characters from both ends up to the middle of the string ($s.size()/2$). Finally, it performs a direct string equality check against t .